

Mulungushi University

School of Engineering and Technology — Department of Computer Science and IT

ICT461: Web Standards and HTTP Fundamentals

Anthony Mfula

Task 2, Part 3 — Diagnostic /inspect Route: Lab Report

Course registration portal — HTTP contract testing

1. Objective

This section of the lab adds a separate diagnostic route, GET/POST /inspect, to the Express API. Its purpose is to echo back exactly what the server received for a request — method, path, headers and body — so that the effect of different Content-Type and Accept headers, different body-encoding formats, and different URL components can be observed directly rather than inferred. Four experiments were carried out: (A) a JSON request, (B) a form-urlencoded request, (C) a direct comparison of the Accept and Content-Type headers, and (D) an inspection of URL anatomy including the behaviour of a URL fragment.

2. Implementation Summary

The /inspect route was added to server.js as its own endpoint, separate from the /api/registrations routes used for the actual registration logic. It uses both of Express's built-in body parsers so that it can correctly interpret either request format:

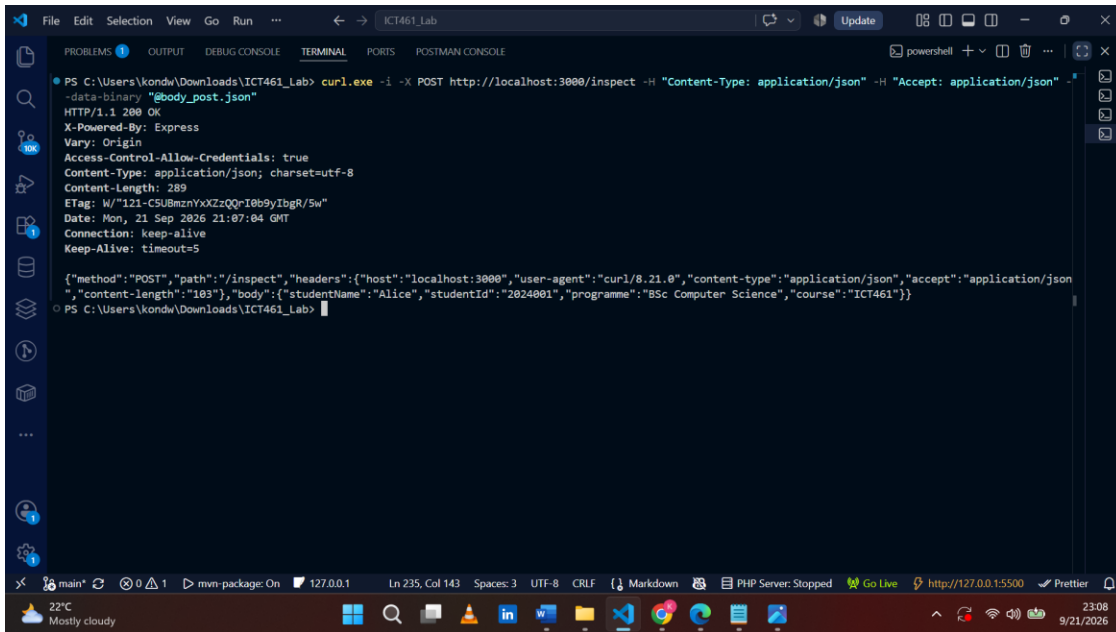
```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.all('/inspect', (req, res) => {
  res.json({
    method: req.method,
    path: req.path,
    headers: req.headers,
    body: req.body
  });
});
```

Because express.json() parses bodies whose Content-Type is application/json, and express.urlencoded() parses application/x-www-form-urlencoded bodies, the same /inspect handler transparently supports both formats used in the experiments below — the correct parser is chosen automatically by Express based on the Content-Type header of the incoming request.

3. Experiment A — Sending JSON to /inspect

A POST request was sent using cURL with Content-Type: application/json and a JSON body containing a sample student registration (name, student ID, programme and course).



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X POST http://localhost:3000/inspect -H "Content-Type: application/json" -H "Accept: application/json" --data-binary '{"method":"POST","path":"/inspect","headers":{"host":"localhost:3000","user-agent":"curl/8.21.0","content-type":"application/json","accept":"application/json","content-length":"103"},"body":{"studentName":"Alice","studentId":"2024001","programme":"BSc Computer Science","course":"ICT461"}}'
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 289
ETag: W/"121-C5UBmznYxXZzQQr-I0b9yIbGR/5w"
Date: Mon, 21 Sep 2026 21:07:04 GMT
Connection: keep-alive
Keep-Alive: timeout=5

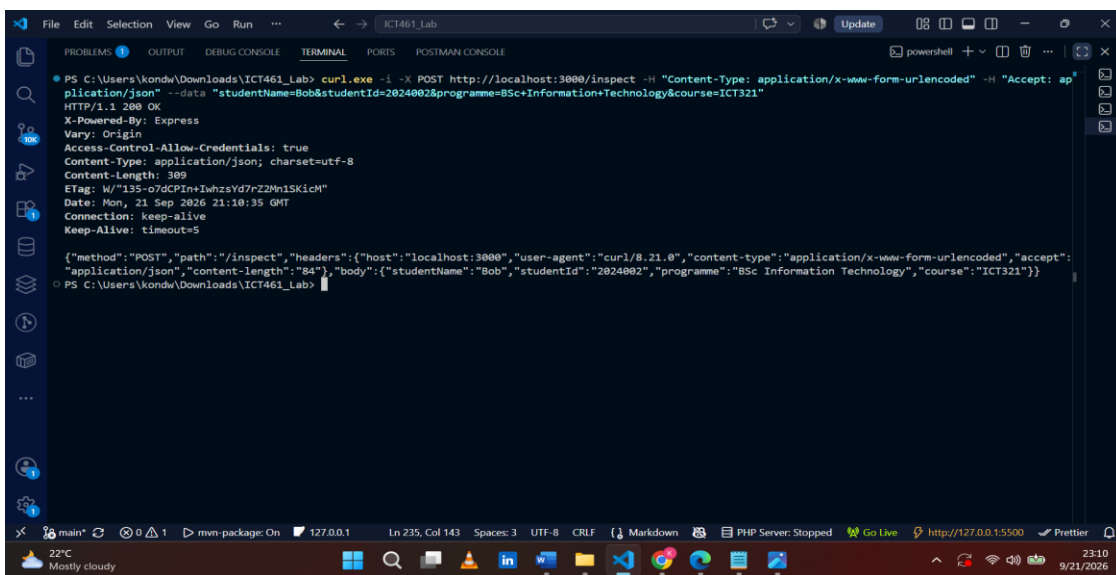
{"method":"POST","path":"/inspect","headers":{"host":"localhost:3000","user-agent":"curl/8.21.0","content-type":"application/json","accept":"application/json","content-length":"103"},"body":{"studentName":"Alice","studentId":"2024001","programme":"BSc Computer Science","course":"ICT461"}}
```

Figure 1: *cURL POST to /inspect with a JSON body, showing the echoed method, headers, and parsed body.*

Result: the server responded 200 OK and echoed back method: "POST", the full request headers (including content-type: application/json and accept: application/json), and a body object whose fields matched the JSON that was sent exactly. This confirms that `express.json()` correctly parsed the raw JSON text into a JavaScript object accessible at `req.body`, and that the `/inspect` route is a faithful mirror of what Express actually received — nothing was silently dropped, renamed, or reformatted.

4. Experiment B — Sending Form-Urlencoded Data to /inspect

The same conceptual registration data was resent, but this time as `application/x-www-form-urlencoded`, with the body written in the `key=value&key=value` cURL `--data` format instead of JSON text.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X POST http://localhost:3000/inspect -H "Content-Type: application/x-www-form-urlencoded" -H "Accept: application/json" --data "studentName=Bob&studentId=2024002&programme=BSc+Information+Technology&course=ICT321"
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 309
ETag: W/"135-07dCPIn-IwhzzYd7r-Z2Wn1SKicM"
Date: Mon, 21 Sep 2026 21:10:35 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"method":"POST","path":"/inspect","headers":{"host":"localhost:3000","user-agent":"curl/8.21.0","content-type":"application/x-www-form-urlencoded","accept":"application/json","content-length":"84"},"body":{"studentName":"Bob","studentId":"2024002","programme":"BSc Information Technology","course":"ICT321"}}
```

Figure 2: *cURL POST to /inspect with a form-urlencoded body, showing correct parsing into the same body shape.*

Result: the response again showed 200 OK, and req.body contained the same four fields (studentName, studentId, programme, course) as plain strings, despite the very different wire format of the request (an encoded query-string-like body rather than JSON text, with spaces sent as + characters, e.g. BSc+Information+Technology). This demonstrates that `express.urlencoded({ extended: true })` correctly decoded the percent/plus-encoded form body into an equivalent object, and that the request headers correctly reported content-type: application/x-www-form-urlencoded so that Express selected the matching parser rather than the JSON one.

5. Experiment C — Comparing Accept and Content-Type

A third request reused the JSON body from Experiment A but set a different Accept header (text/html, application/xml) while keeping Content-Type: application/json for the request body itself. This isolates the two headers, which are easy to confuse because they look similar but describe opposite directions of the conversation.

```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X POST http://localhost:3000/inspect -H "Content-Type: application/json" -H "Accept: text/html,application/xml" --data-binary "@body_post.json"
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 298
ETag: W/"12a-JvTGksllhLQRuSyAHGLGo#isTho"
Date: Mon, 21 Sep 2026 21:12:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"method":"POST","path":"/inspect","headers":{"host":"localhost:3000","user-agent":"curl/8.21.0","content-type":"application/json","accept":"text/html,application/xml","content-length":"183"},"body":{"studentName":"Alice","studentId":"12024001","programme":"BSc Computer Science","course":"ICT461"}}
```

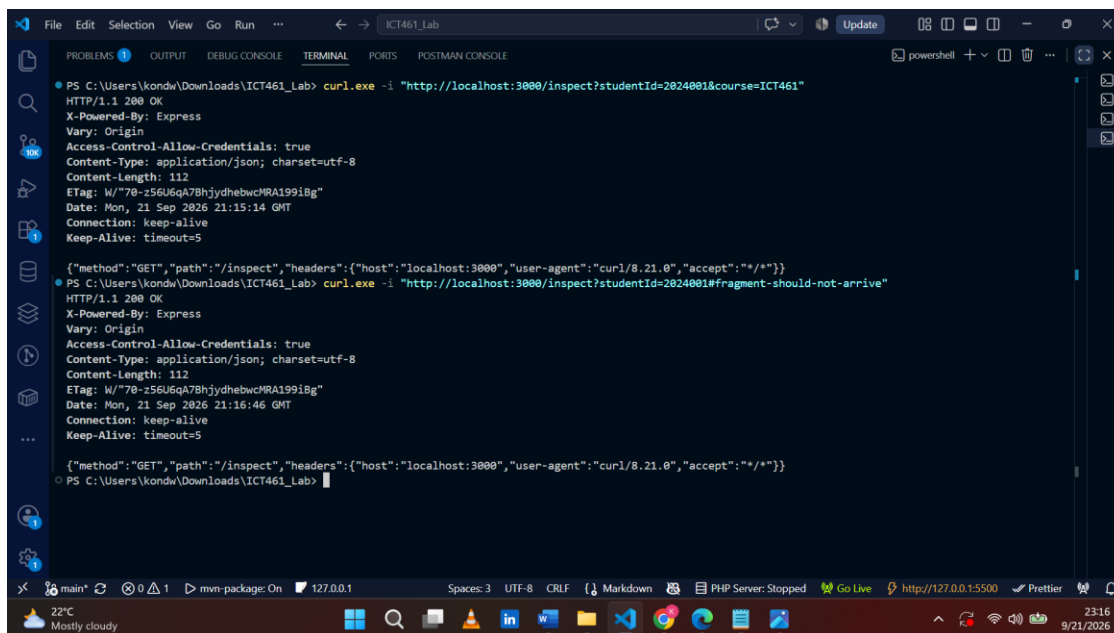
Figure 3: Content-Type describes the outgoing request body; Accept expresses a preference for the response format.

Header	Direction / Meaning	Value sent in this experiment
Content-Type	Request header describing the media type of the body the client is sending.	application/json
Accept	Request header telling the server which media type(s) the client would prefer in the response.	text/html, application/xml
Actual response Content-Type	What the server chose to return, regardless of the client's Accept preference.	application/json; charset=utf-8

Result: even though the client's Accept header asked for text/html or application/xml, the response Content-Type was still application/json; charset=utf-8, because the /inspect handler always calls `res.json()` regardless of what the client requested. This is expected for this lab's diagnostic route — it confirms Content-Type describes the format of the data actually being sent in a message body, while Accept only expresses the client's preference for the response and does not force the server to comply with it. A production API practising real content negotiation would inspect `req.headers.accept` and choose its response format (or reply 406 Not Acceptable) accordingly; this prototype intentionally always returns JSON for simplicity.

6. Experiment D — URL Anatomy and the Fragment Test

Two GET requests were sent to /inspect to examine how a request URL is broken down and transmitted. The first used a normal query string; the second appended a #fragment to the same URL.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i "http://localhost:3000/inspect?studentId=2024001&course=ICT461"
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 112
ETag: W/"70-z56U6qA78hjdhebwcmRA199iBg"
Date: Mon, 21 Sep 2026 21:15:14 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"method":"GET","path":"/inspect","headers":{"host":"localhost:3000","user-agent":"curl/8.21.0","accept":"*//*"}}
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i "http://localhost:3000/inspect?studentId=2024001#fragment-should-not-arrive"
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 112
ETag: W/"70-z56U6qA78hjdhebwcmRA199iBg"
Date: Mon, 21 Sep 2026 21:16:46 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"method":"GET","path":"/inspect","headers":{"host":"localhost:3000","user-agent":"curl/8.21.0","accept":"*//*"}}
```

Figure 4: Two GET requests to /inspect — a plain query string, and the same URL with a #fragment appended.

Component	Scheme	Host	Port	Path + Query
Request 1	http	localhost	3000	/inspect?studentId=2024001&course=ICT461
Request 2	http	localhost	3000	/inspect?studentId=2024001 (+ #fragment-should-not-arrive, client-side only)

Result: both requests returned an identical echoed body — `{"method":"GET","path":"/inspect","headers":{"...}}` — with no trace of the query string or the fragment in `req.path` (Express separates query parameters out of `req.path` by design), and, critically, no trace whatsoever of `#fragment-should-not-arrive` anywhere in the echoed request. This confirms the standard rule that a URL fragment (the part after `#`) is a purely client-side addressing mechanism, interpreted only by the browser (e.g. to scroll to an element or drive client-side routing) and is never transmitted to the server as part of the HTTP request line or headers.

7. Conclusions

- The /inspect route works as a reliable diagnostic tool: it echoes method, path, headers and body without modification, so any mismatch between what a client believes it sent and what the server received can be caught immediately.
- Express automatically selects the correct body parser (json vs urlencoded) based on the Content-Type header, and both parsers produced an equivalent `req.body` object from differently-encoded input.
- Content-Type and Accept are easily confused but describe opposite directions of the request/response exchange: Content-Type describes the body being sent; Accept is only a preference for the reply and does not obligate the server to honour it.
- A request URL's scheme, host, port, path and query string are all sent to the server, but a #fragment is never transmitted — it is resolved entirely within the browser.